pcapML: Making Network Traffic Datasets Reproducible

Jordan Holland
Princeton University
Princeton, New Jersey, USA

Jess Alencaster
California Polytechnic State University
San Luis Obispo, California, USA

Nick Feamster
University of Chicago
Chicago, Illinois, USA

Paul Schmitt
California Polytechnic State University
San Luis Obispo, California, USA

Abstract

Machine learning for network traffic analysis depends on labeled data, but the workflows used to construct training pipelines and evaluation datasets are often hard to reproduce. Labels are commonly stored outside the packet trace and attached later through dataset-specific reconstruction, thus the same raw traffic can yield different labeled datasets. We present pcapML, a system for constructing reproducible labeled network traffic datasets in PcapNG. pcapML stores label metadata inline with the packet trace so that the trace itself becomes the full dataset, rather than a packet file plus a separate reconstruction procedure. For new traffic, pcapML can assign labels during capture using eBPF-based process attribution in the Linux kernel. For existing datasets, it can ingest an explicit external label mapping and convert that mapping into the same packet-labeled format. We compare pcapML against conventional post-hoc labeling workflows and show that the differences are measurable even in controlled collection settings. The result is a common labeled trace format that makes dataset construction more reproducible and lets different studies build from an identical foundation.

CCS Concepts

• **Networks** → **Network measurement**; **Network performance analysis**; • **Computing methodologies** → **Machine learning**.

Keywords

Network traffic labeling, Traffic classification, Dataset curation

ACM Reference Format:
Jordan Holland, Jess Alencaster, Nick Feamster, and Paul Schmitt. 2026. pcapML: Making Network Traffic Datasets Reproducible. In *ACM/IRTF Applied Networking Research Workshop (ANRW '26), July 20, 2026, Vienna, Austria.* ACM, New York, NY, USA, 7 pages. https://doi.org/10.1145/3822163.3827923

1 Introduction

Traffic classification is a fundamental requirement of many network management and security problems, and has been a focus of our field for decades [26]. It underpins tasks such as application detection, device identification, intrusion detection, and website fingerprinting [5, 20, 23, 28]. The increasing complexity of networked systems and protocols, the encryption of traffic payloads, and the availability of large packet traces have made machine learning attractive, and often the only practical approach, for these tasks.

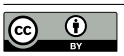
Much of this work is hard to compare or reproduce because the data pipeline is underspecified. Researchers routinely build custom collection, preprocessing, and labeling workflows, and even basic terms such as "flow" and "application" are used differently across papers. The end result is that two studies can report results on what looks like the same dataset, yet be evaluating completely different tasks.

The same raw traffic often turns into different datasets across papers. One work may define a flow as a 4-tuple, another as a 5-tuple, and others may treat the same 5-tuple as either uni-directional or bi-directional. Some benchmarks are released as raw packet traces with metadata in separate files, others as pre-extracted features, and many require undocumented reconstruction before they can be used.

We have found that labeling is a particularly weak point in this pipeline. In many datasets, labels are stored separately from the packet trace, so producing a benchmark requires dataset-specific reconstruction and interpretation. Different researchers can start from the same raw traffic and still produce different labeled datasets. This critique applies to our own work as well.

The root cause is a mismatch between where labels are *created* and where they are *attached*. In most existing workflows, traffic is captured first and labeled later through joins over timestamps, 5-tuples, filenames, or other metadata. That step is ambiguous, and small differences in implementation are enough to produce different datasets from the same trace. §2 shows that this is standard practice across widely used benchmarks. Earlier workflows often could not observe process identity at packet-capture time, so labels were reconstructed later from coarser metadata.

We present pcapML [35], a system for constructing reproducible labeled network traffic datasets using PcapNG, a widely supported packet capture format. pcapML records label metadata inline with the packet trace so that the trace itself becomes the full dataset, reducing dependence on brittle reconstruction pipelines. pcapML can also assign labels during capture using eBPF-based process attribution in the kernel for new traffic captures. For existing datasets, it can ingest an explicit external label mapping and convert that chosen mapping into the same normalized representation. The mechanisms in pcapML are mostly familiar; the contribution is a practical way to produce canonical labeled packet traces.



ANRW '26, Vienna, Austria

ACM ISBN 979-8-4007-2876-1/2026/07
https://doi.org/10.1145/3822163.3827923

Table 1: Work stemming from the same dataset ends up testing methods on differing versions of the same dataset, rendering direct comparisons impossible. (♦ denotes merged classes.)

			Class Distribution														
				DoS										Web			
Source	Year	Benign	Bot	DDoS	GoldenEye	Hulk	Slowhttptest	Slowloris	Heartbleed	Infiltration	PortScan	FTP-Patator	SSH-Patator	Brute Force	SQL Injection	XSS	
Original Dataset	2018	2,273,097	1,966	128,027	10,293	231,073	5,499	5,796	11	36	158,930	3,938	5,879	1,507	21	652	
Vinakayumar	2019	80,000	1,966	8,000		8,000♦			-	-	8,000	7,938	5,879	2,180♦			
Zhang	2019	339,621	1,441	16,050	7,458	14,108	4,216	3,869	-	-	158,673	3,907	2,511	1,353	12	631	
Zhou et al.	2020	439,683	-	-	10,293	230,124	5,499	5,796	11	-	-	-	-	-	-	-	
Barut	2020	248,067	-	45,168		29,754♦			-	66,914	153,028	3,958	2,464	2,019♦			
Stiawan	2020	454,396	367	76,265♦					-	6	32,882	2,717	-	426	-	-	

This makes it possible to compare results on identically labeled traces, rather than on different reconstructions of the same raw traffic. Prior work has largely focused on reproducible models or standardized feature sets. pcapML instead gives researchers a common representation for labeled traffic traces. It preserves the raw packets while making labels and their semantics explicit in the trace itself, whether the data was collected live or imported from an existing benchmark.

This paper makes three contributions: 1) we analyze the reproducibility problem in network traffic ML and show, through concrete dataset examples, how post-hoc labeling produces incompatible benchmarks (§2); 2) we design and implement pcapML, a PcapNG-based framework for constructing self-contained labeled traffic datasets, including both kernel-level attribution for live captures and normalization of existing datasets (§3); and 3) we evaluate pcapML against conventional post-hoc labeling workflows and quantify the labeling error that current practice introduces (§4).

2 The Reproducibility Problem

We examine four widely used network traffic datasets and how they are used in the literature. In each case, papers derive materially different labeled versions of the same underlying data. This makes cross-paper comparison, and more importantly reproducibility, unreliable.

These examples are not meant to single out papers. If anything, the papers we cite set a high bar for methodological transparency: they report their preprocessing choices clearly enough for the differences to be visible. Less transparent work often cannot be evaluated at this level. The problem we highlight here is *structural*: when labels are attached after capture through post-hoc procedures, divergence is difficult to avoid.

2.1 CICIDS 2017 Dataset

The CICIDS 2017 intrusion detection dataset [32] contains five days of traffic across 15 classes. It was released in two forms: a pre-extracted feature set and the raw packet traces. The raw release consists of one or two PCAP files per day along with a CSV that links flows to labels through flow timestamps and 5-tuples. That release format leaves room for divergence: the benchmark feature set is one interpretation of the raw traffic, and later papers can reconstruct the raw traces differently. The unit of analysis also varies across work. The original release operationalizes flows through `CICFlowMeter`, which constructs biflows, while later work using the same raw traffic can parse flows differently [32, 42].

Table 1 documents the variation. Papers merge classes, drop rare attacks, or reduce the task to binary classification [38]. Engelen *et al.* also audit the dataset itself and find that more than 20% of the original traces require reconstruction or relabeling [12]. They trace the problem to errors in traffic generation, flow construction, feature extraction, and labeling, including errors introduced by the timestamp-based join between attack schedules and flow records [12]. Subsequent work often continues to benchmark on the original labels. In practice, CICIDS 2017 functions less as a single benchmark dataset than as a family of related but incompatible variants.

2.2 VPN-nonVPN Dataset

The ISCX VPN-nonVPN dataset [11] contains traffic from 7 application types, each captured with and without VPN encryption, yielding 14 classes. It is released as a set of PCAP files, with application metadata encoded in filenames such as `email1a.pcap`. Those filenames identify specific applications, but the dataset does not provide a mapping from individual files to the broader label set used in evaluation.

Wang *et al.* report that they could not fully reproduce the dataset after contacting the authors, and therefore use 12 classes instead of 14 [40]. They also note that some files "can be labeled as either 'Browser' or 'Streaming'," and conclude "We can't solve this problem" [40]. Lotfollahi *et al.* explicitly "redefine the labels" for different tasks, yielding 12- and 17-class variants from the same raw traffic [22]. Shapira and Shavitt collapse the dataset to five broad categories [31], while Barut *et al.* derive 7-, 18-, and 31-class versions from the same source files [4]. Because the dataset does not enforce a single mapping from traces to labels, papers often arrive at different interpretations. Across the literature, we find the number of classes associated with this dataset ranges from **2 to 31**.

2.3 UNSW-NB15 Dataset

The UNSW-NB15 dataset [25] shows that the problem is not limited to labels. Here the divergence arises after labeling. Its raw traffic and derived feature releases make it possible to compare nominally similar results that are based on different task definitions and processing pipelines. Binary and multi-class framing change reported accuracy by 12 percentage points on the same model [24]. Papers also handle class imbalance differently, using resampling strategies such as SMOTE or hybrid sampling [15, 27]. Sarhan *et al.* show that re-extracting features from the same raw traffic can shift minority-class F1 from 0.09 to 0.69 [30]. Authors may therefore begin with the same PCAPs and still end up evaluating different datasets.

2.4 DARPA 1998 / NSL-KDD Datasets

The DARPA 1998 intrusion detection dataset is an early and still influential benchmark [20]. Later benchmark families such as KDD'99 and NSL-KDD descend from this line of work, but the reconstruction

Table 2: Usage of the KDD-CUP98 dataset differs from both the original release and other works.

		Class Distribution				
Source	Year	Normal	DoS	Probe	R2L	U2R
Dataset Release [20]	2000	1,157,873	1,919,937	54,793	949	48
Khan *et al.* [17]	2007	878,318	308,808	15,120	3,327	39
Wang *et al.* [39]	2017	1,309,598	2,152,850	89,301	14,535	436

problem begins earlier, at the raw DARPA 1998 release itself. It was created to support comparison across intrusion detection methods, with seven weeks of training data and two weeks of test data containing benign traffic and multiple attack types. The dataset ships raw PCAP files and separate session-label files, so reconstruction requires a non-trivial preprocessing pipeline. The raw traffic was released as daily packet traces, resulting in 45 PCAP files, with separate "list" files containing labels for TCP and UDP sessions. Each record includes a start date and time, a 4-tuple, and a session label. That format leaves the final mapping from labels to packets to whoever rebuilds the dataset. Wang *et al.* describe the difficulties [39]:

> "The traffic format of DARPA1998 is non-split pcap, which must be split into multiple network flow files. In addition, the label files contain a few problems, such as duplicated records and incorrect labels. For example, the label file 'Test/Week2/Friday' contains a record of '07/32/1998,' which is an obvious date error. Therefore, the dataset requires preprocessing before the experiments can be conducted."

Table 2 shows that papers reconstruct different versions of the dataset from the same raw release. The class counts reported by Khan *et al.* and Wang *et al.* differ from the original release and from each other [17, 39]. A recent survey reports variants ranging from 2 classes to 23 [21]. Many papers also replace the intended KDDTest+ protocol with random partitioning, changing the task from open-world to closed-world classification [13].

2.5 Common Sources of Divergence

Taken together, these examples highlight recurring sources of divergence and show why those differences persist across the literature. Researchers may begin from the same raw traffic and still arrive at different benchmarks for two different reasons: they may reconstruct labels and the unit of analysis differently from under-specified releases, or they may make different choices about task definition and preprocessing.

Terminology and units of analysis. The datasets examined above describe network traffic using terms common throughout the field ("sessions," "flows," "applications"), yet these terms carry different definitions across papers. DARPA 1998 focuses on TCP/UDP "sessions"; CICIDS 2017 operationalizes flows through `CICFlowMeter`; the VPN-nonVPN dataset treats single flows as "applications." The term "flow" itself varies across the literature: some work uses 4-tuples, others use 5-tuples, and both can be defined in uni-directional or bi-directional ways [42]. Different flow timeout values also change classification accuracy by up to 3% [11]. Without standard definitions of the unit of analysis, papers that appear comparable may be *addressing different problems.*

Task definitions. The choice between binary and multi-class framing is often buried in a paper's methodology section, yet it changes reported accuracy by 12–23 percentage points on the same model and data [24]. Without standardized task definitions, headline accuracy numbers say very little about cross-paper performance. Such task choices are deliberate research decisions that pcapML does not resolve.

Preprocessing pipelines. Engelen *et al.* document incorrect labels, while Sarhan *et al.* show that even with correct labels, different feature extraction pipelines produce incomparable datasets from the same raw traffic. Labeling, flow construction, and feature extraction can each introduce divergence, but they do so at different stages of the pipeline. pcapML addresses the labeling axis. Feature standardization efforts such as Sarhan *et al.*'s address a later stage of the problem. Both matter, but labeling comes first: it defines the foundation for later stages, thus ambiguity at the labeling stage propagates through the rest of the pipeline.

These sources of divergence do not necessarily narrow over time. Engelen *et al.* find that over 20% of CICIDS 2017 labels are incorrect [12], yet much of the subsequent work continues to use the original labels. One reason is comparability. Using corrected labels can make results less directly comparable with prior work that used the original dataset, so established versions of a benchmark may continue to be used.

Section 3 presents pcapML, which addresses the unit-of-analysis and labeling stages by embedding labels and packet-to-sample relationships in the trace itself. It does not standardize the downstream task definition. It does create a stable foundation for comparing downstream layers. Section 4 measures the error introduced by current practice.

3 pcapML

pcapML uses a single output format and we provide two ways to produce it. It stores labels and packet-to-sample relationships directly in PcapNG. New datasets can be created through live capture with eBPF-based process attribution, and existing datasets can be converted offline into the same form. Live capture and offline conversion serve different roles. Live capture records new process-attribution labels as packets are observed. Offline conversion does not recover missing ground truth; it preserves an existing reconstruction in the same packet-labeled format.

3.1 Labeled Trace Format

Output file format. We use PcapNG because the format supports per-packet metadata through block options and is widely supported by common packet capture and analysis tools, including `Wireshark`, `tshark`, `libpcap`, and `tcpdump`[34, 41]. pcapML uses the built-in `opt_comment` option to encode labels directly in the trace, so packet data and label metadata stay in the same file.

SampleIDs. Across traffic analysis tasks, each labeled example corresponds to one or more packets. We use this observation to define

sampleIDs. A sampleID marks a packet group without committing to a task-specific notion of sample. Storing a sampleID with every packet preserves the grouping information needed downstream. It also makes later dataset splits explicit. Papers can report train/test partitions in terms of sampleIDs rather than reconstructing them from filenames or preprocessing code.

During live capture, pcapML assigns sampleIDs based on normalized 5-tuples (source IP, destination IP, source port, destination port, protocol), with addresses ordered so that both directions of a bidirectional flow share the same sampleID. Each packet's comment field then records the sampleID, the process name, the packet direction (derived from the capture path and local network configuration), and, when available from DNS or SNI, a destination label. We encode this metadata as a simple key-value record in `opt_comment`, so the format remains extensible: future tools can add new fields while existing consumers continue to use the fields they recognize:

```
s=3,proc=Chrome_ChildIOT,dir=wan2lan,dst=clients2.google.com
s=7,proc=Chrome_ChildIOT,dir=lan2wan,dst=accounts.google.com
s=10,proc=Chrome_ChildIOT,dir=wan2lan,dst=www.amazon.com
```

3.2 eBPF-Based Live Capture

The core of pcapML is a live capture engine that uses eBPF for kernel-level process attribution. Instead of recovering process identity later from external metadata, pcapML records the responsible process when the kernel handles the packet.

Architecture. pcapML attaches eBPF programs at two layers of the Linux network stack: socket-to-process mapping and packet capture. For socket-to-process mapping, it uses kprobes on `tcp_connect`, `inet_csk_accept`, `tcp_close`, `udp_sendmsg`, and `udp_destroy_sock`. These probes record the mapping between a socket's endpoint information and the owning process. Userspace later combines that information with the transport protocol to assign normalized 5-tuple sampleIDs.

For packet capture, pcapML attaches `cgroup_skb` programs to a selected cgroup for both ingress and egress, using the root cgroup by default. When a packet traverses the cgroup socket buffer, the eBPF program looks up the associated process, copies the packet data and metadata into a BPF ring buffer, and delivers it to userspace. Restricting capture to a selected cgroup is useful in containerized or multi-tenant deployments where one may want to label traffic for a single service without capturing the rest of the host. pcapML's design allows it to be applied to traffic anywhere along the path, including at gateways. Of course, host-local process information is not available at middleboxes; however, pcapML still captures direction and destination labels.

Userspace reads events from the ring buffer, computes wall-clock timestamps from the kernel's monotonic clock, assigns sampleIDs based on normalized 5-tuples, and writes packets to the output PcapNG file with the packet labels encoded in the comment field.

Why eBPF. Userspace approaches such as `libpcap` combined with polling of `/proc` or `lsof` are prone to timing errors: short-lived connections can begin and end between observation points, so the recovered labels are only best-effort. eBPF executes in the kernel packet path, which lets pcapML capture process identity and packet data at the same time.

3.3 Offline Tools

pcapML also provides an offline path for datasets that are released as raw traces plus metadata. The tools cover the operations needed to bring existing traces into the same pcapML format. This matters because collecting new labeled traffic is often expensive or infeasible: many studies must start from existing packet traces, and pcapML provides a way to relabel those traces on a more explicit and reproducible foundation.

The `label` tool applies labels to an existing PCAP using external metadata and produces a labeled PcapNG trace; each label entry combines a timestamp window with a BPF expression, so a packet must satisfy both to match, reducing the ambiguity of timestamp-only joins. This offline path does not recover process identity or other ground truth that was not captured in the original trace. Rather, it records the selected reconstruction in the trace so later users do not have to rediscover or reimplement it. For example, in VPN-nonVPN, papers disagree on how filename-level metadata maps to 12-, 14-, 17-, or 31-class tasks. pcapML does not decide which mapping is correct; it lets a curator encode one chosen mapping as a packet-labeled trace, so later studies can consume the same artifact instead of rebuilding that mapping from filenames. Alternative task definitions can still exist, but they become explicit dataset versions rather than implicit reconstruction choices. The `split` tool breaks a labeled PcapNG into individual PCAP files, one per sampleID. The `sort` tool reorders packets in a labeled PcapNG by sampleID and timestamp. The `strip` tool removes labels from a PcapNG file and produces a plain PCAP.

We also provide a Python package, installable as `pip install pcapml`, for consuming pcapML traces in analysis pipelines. The wrapper parses pcapML PcapNG comments, exposes packets with their sampleIDs and labels, and can load dataset metadata into pandas DataFrames with labels used as column names. This lets users work with pcapML datasets without reimplementing PcapNG parsing or label extraction.

4 Evaluation

We evaluate pcapML in a small controlled collection with two goals. First, we want to see which traffic distinctions process labels and destination labels can recover. Second, we use those labels to measure the error introduced by timestamp-based attribution. The experiment records the same traffic at two vantage points. The host capture contains 430,969 packets across five sessions.

4.1 Experimental Setup

We run five traffic generators sequentially on a Linux desktop, each in isolation for approximately 300 seconds:

- **chrome_browse**: Selenium-driven Chrome visiting diverse websites (Amazon, NYT, GitHub, BBC, Reddit, etc.)
- **chrome_stream**: Selenium-driven Chrome streaming YouTube videos
- **firefox_browse**: Selenium-driven Firefox visiting the same URL list as Chrome
- **ssh**: Interactive SSH session to a remote host
- **rsync**: Bidirectional file sync over SSH to a remote host

Two simultaneous pcapML captures record all traffic: a *host capture* (eBPF cgroup mode, snap length 120 B, process + destination labels) and a *gateway capture* (eBPF TC mode on an OpenWrt router's

Table 3: Host destination label profiles. Chrome browsing produces a long tail of 526 domains; streaming concentrates on YouTube CDN.

Generator	Unique Dst	Top Destinations
chrome_browse	526	g1.nyt.com, edgedl.me.gvt1.com, ot.www.cloudflare.com, diverse web/ad-tech
chrome_stream	35	*.googlevideo.com CDN, www.youtube.com, i.ytimg.com

WAN interface, snap length 400 B, destination labels only). This gives us the same traffic with two different levels of visibility. At the host, pcapML can attach both process labels and destination labels to each packet. At the gateway, the same traffic is visible later in the path, but process context is gone and only destination labels remain. That distinction matters because many post-hoc labeling workflows operate from gateway-style observations rather than from host-local process metadata.

The different snap lengths do not affect process attribution, which comes from kernel socket context. They can affect destination-label recovery from packet contents: the gateway uses a longer snap length so pcapML can observe DNS and TLS ClientHello metadata needed for destination labeling.

Within that setup, the traffic generators form three comparison pairs. The Chrome browse/stream pair asks whether destination labels can separate two sessions that come from the same browser process. The Chrome/Firefox pair asks the opposite question: when two sessions visit the same sites, can host-local process labels still separate them? The ssh/rsync pair is different. Rsync runs through `ssh` to the same remote host, so the packets look the same at both label layers: the process is `ssh`, and the destination is the same SSH server. In that case, neither process labels nor destination labels provide enough information to tell the two activities apart. These three cases let us compare what each vantage point recovers from the same traffic.

4.2 Destination Labels for Chrome Sessions

Table 3 shows that destination labels cleanly separate the Chrome pair. The browsing session contacts 526 diverse domains spanning news sites, ad exchanges, and analytics trackers. The streaming session contacts only 35 domains, with bulk traffic going to `*.googlevideo.com` CDN edge servers. At the host, process labels do not separate these sessions: both are dominated by `Chrome_ChildIOT`. The gateway sees the same destination pattern independently: 484 unique destinations for browsing and 34 for streaming.

4.3 Process Labels for Browser Sessions

Chrome browse and Firefox browse visit the *same URL list*, producing heavily overlapping destination profiles. Both sessions' top destinations include `nytimes.com`, `cloudflare.com`, `reddit.com`, and similar sites. Destination labels do not separate these sessions, but the host capture does: Chrome is dominated by `Chrome_ChildIOT`, whereas Firefox is dominated by `Socket Thread`. Firefox also exposes its internal thread architecture: 17 distinct `DNS Resolver` threads (4,259 packets), `selenium-manage` (1,507 packets), and `webdriver` infrastructure threads (976 packets).

4.4 Tunneled Protocols

The ssh/rsync pair is useful for a different reason: it shows a case that pcapML does not resolve. Neither label type disambiguates ssh from rsync. Both sessions connect to the same remote host over SSH. There is no DNS query, no TLS ClientHello, and no distinct process identity because rsync tunnels through `ssh`. Both are labeled `ssh` at the process layer, and destination labeling collapses both sessions to the same unlabeled SSH server. Every packet is identically labeled across both sessions at both vantage points.

That limit is inherent to the traffic, not specific to pcapML. SSH tunnels are opaque to passive observation. pcapML does not remove that limitation; it preserves it in the trace. Both labels reduce to "`ssh` to an unlabeled destination," which makes clear that the packets cannot be separated further based on packet evidence alone. A timestamp-based workflow would still assign them a class, but would not reveal that the assignment rests on an assumption rather than on packet evidence.

4.5 Timestamp-Based Mislabeling

Using pcapML's ground-truth process labels, we count how many non-experiment packets fall within each generator's time window. A timestamp-based labeler would attribute all of these to the active generator. Table 4 shows the results.

The firefox_browse window has the highest contamination (5.0%, 5,992 packets), primarily from Firefox's own infrastructure: 17 `DNS Resolver` threads (4,259 packets) and webdriver threads (976 packets). This case shows a subtler issue than simple contamination. Some packets, such as those from Firefox DNS resolver threads, are part of the browsing workload and may reasonably be retained. Others, such as webdriver and `python3` traffic, are artifacts of the automation harness and should likely be excluded. Process-level labels preserve that distinction; timestamp-based labels erase it.

The aggregate 1.4% mislabel rate understates the problem. Our experiment is intentionally controlled: generators run sequentially on a single desktop with limited competing activity. Real user environments include system services, chat clients, background synchronization, and browser helper processes. Even in this favorable setting, timestamp-based attribution reaches 5% error for Firefox.

Together, these cases show why the vantage point matters. The host can retain process context that is unavailable at the gateway, while the gateway still reflects the destination-level information used in many post-hoc pipelines. Neither view is complete on its own, and some traffic remains ambiguous at both. What pcapML changes is that the labels present at each vantage point are attached to packets at collection time rather than reconstructed later.

4.6 Limitations

Our experiment uses automated generators on a single Linux host, so it cannot capture the full background noise or behavioral diversity of a real multi-user environment. The gateway labels depend on

Table 4: Timestamp mislabel analysis. A naive labeler that attributes all packets in a time window would mislabel 6,028 packets. Firefox's 5% rate is driven by its multi-threaded DNS resolver and webdriver infrastructure.

Generator	Total	Mislabeled	Rate	Key Background
chrome_browse	222,707	30	0.01%	curl, python3
chrome_stream	78,694	6	0.01%	python3
firefox_browse	119,853	5,992	**5.0%**	DNS Resolver (17 threads), webdriver
ssh	2,643	0	0%	—
rsync	7,072	0	0%	—
Total	**430,969**	**6,028**	**1.4%**	

DNS and TLS SNI, both of which become less informative under encrypted DNS or encrypted SNI/ClientHello. In that case, pcapML records the missing destination label rather than inferring one, so the trace preserves the loss of evidence explicitly. Finally, our current implementation targets Linux; extending the approach to macOS or Windows would require different kernel instrumentation.

5 Related Work

Reproducibility and network traffic datasets. Irreproducibility has been documented across many applied ML domains [10, 16, 29]. In network traffic analysis, a large part of the problem lies in how datasets are constructed and released. Many benchmarks separate the raw packet trace from the labels needed to interpret it, leaving researchers to reconstruct the dataset through joins over timestamps, flow keys, filenames, or other metadata. Arp *et al.* show that preprocessing decisions can invalidate security ML results [2]. Engelen *et al.* document dataset defects and relabeling issues in CICIDS 2017 [12], while Sarhan *et al.* show that standardizing feature extraction produces materially different results from the same raw traffic [30]. Here, the problem starts earlier, at the stages where traffic is grouped, labeled, and turned into a dataset.

Traffic labeling approaches. DPI-based tools such as L7-filter [18] infer labels from payload content, but these approaches are limited by the increasing use of encryption. DNS/SNI-based approaches associate flows with destinations through DNS lookups or TLS ClientHello metadata [7], but they do not provide process identity and degrade under encrypted DNS or encrypted SNI/ClientHello [8, 33]. NetFlow/IPFIX exports support flow-level analysis without process attribution [1]. Manual labeling remains common and is error-prone, as discussed in Section 2. pcapML is complementary to these approaches. Its contribution is not a new traffic classifier, but a way to record labels and packet groupings directly in the trace using capture-time process attribution or offline conversion from existing metadata.

eBPF-based network observability. Several tools use eBPF for network monitoring. Cilium/Hubble provides flow-level observability with process context for Kubernetes environments [9]; Pixie captures protocol-level data with process attribution [3]; bpftrace and BCC provide general tracing infrastructure [6, 14]. These systems target system monitoring, not dataset construction. pcapML uses the same kernel-level observability mechanisms for a different purpose: constructing labeled packet traces that can serve as stable ML datasets.

PcapNG format. The PcapNG capture format provides extensible block options that can attach metadata to packet traces. Le *et al.* use PcapNG in AntMonitor to attach mobile application names to raw packets [19]. Velea *et al.* encode pre-processed features (encryption type, protocol, flow packet count) using custom block options [36, 37]. pcapML differs in scope. It does not store task-specific features. Instead, it treats the trace itself as the dataset by encoding labels and packet-to-sample relationships in a form that remains compatible with standard PcapNG tools.

6 Conclusion

Dataset construction in network traffic analysis should be treated as part of the measurement record, not as a separate, error-prone, reconstruction step. A packet trace alone does not fully specify a benchmark if its labels and packet groupings are attached later and left to human interpretation through dataset-specific joins and conventions. In that setting, the same raw traffic can yield different labeled datasets.

pcapML is one way to narrow that source of variation. It records labels and packet-to-sample relationships in the trace itself, either during live capture or by converting existing datasets into the same representation. pcapML does not resolve every source of incomparability; task definition still matters. It does make the labeled trace a more complete and reproducible basis for dataset construction and evaluation.

References

[1] Paul Aitken, Benoît Claise, and Brian Trammell. 2013. Specification of the IP Flow Information Export (IPFIX) Protocol for the Exchange of Flow Information. RFC 7011. doi:10.17487/RFC7011

[2] Daniel Arp, Erwin Quiring, Feargus Pendlebury, Alexander Warnecke, Fabio Pierazzi, Christian Wressnegger, Lorenzo Cavallaro, and Konrad Rieck. 2022. Dos and Don'ts of Machine Learning in Computer Security. In *31st USENIX Security Symposium (USENIX Security 22)*. USENIX Association, Boston, MA, 3971–3988. https://www.usenix.org/conference/usenixsecurity22/presentation/arp

[3] Omid Azizi, Yaxiong Zhao, Ryan Cheng, John P. Stevenson, and Zain Asgar. 2021. Pixie's protocol tracing with eBPF. In *Linux Plumbers Conference.* https://lpc.events/event/11/contributions/945/attachments/896/1750/Paper%20%28PDF%29.pdf

[4] Onur Barut, Yan Luo, Tong Zhang, Weigang Li, and Peilong Li. 2020. Netml: A challenge for network traffic analytics. *arXiv preprint arXiv:2004.13006* (2020).

[5] Laurent Bernaille, Renata Teixeira, and Kave Salamatian. 2006. Early application identification. In *Proceedings of the 2006 ACM CoNEXT conference.* 1–12.

[6] bpftrace Authors. 2026. About bpftrace. https://bpftrace.org/about. Accessed 2026-04-08.

[7] Francesco Bronzino, Paul Schmitt, Sara Ayoubi, Hyojoon Kim, Renata Teixeira, and Nick Feamster. 2021. Traffic Refinery: Cost-Aware Data Representation for Machine Learning on Network Traffic. *Proceedings of the ACM on Measurement and Analysis of Computing Systems* 5, 3, Article 40 (2021), 24 pages. doi:10.1145/3491052

[8] Zimo Chai, Amirhossein Ghafari, and Amir Houmansadr. 2019. On the Importance of Encrypted-SNI (ESNI) to Censorship Circumvention. In *9th USENIX Workshop on Free and Open Communications on the Internet (FOCI 19)*. USENIX Association, Santa Clara, CA. https://www.usenix.org/conference/foci19/presentation/chai

[9] Cilium Authors. 2026. Introduction to Cilium & Hubble. https://docs.cilium.io/en/stable/intro/. Accessed 2026-04-08.

[10] Open Science Collaboration et al. 2015. Estimating the reproducibility of psychological science. *Science* 349, 6251 (2015).

[11] Gerard Draper-Gil, Arash Habibi Lashkari, Mohammad Saiful Islam Mamun, and Ali A Ghorbani. 2016. Characterization of encrypted and vpn traffic using time-related. In *Proceedings of the 2nd international conference on information systems security and privacy (ICISSP)*. 407–414.

[12] Gints Engelen, Vera Rimmer, and Wouter Joosen. 2021. Troubleshooting an Intrusion Detection Dataset: the CICIDS2017 Case Study. In *2021 IEEE Security and Privacy Workshops (SPW)*. IEEE, 13–19.

[13] Jian Gu and Shan Lu. 2021. An Effective Intrusion Detection Approach Using SVM with Naive Bayes Feature Embedding. *Computers & Security* 103 (2021).

[14] iovisor. 2026. BCC: Tools for BPF-based Linux IO analysis, networking, monitoring, and more. https://github.com/iovisor/bcc. Accessed 2026-04-08.

[15] Kewen Jiang, Wenyuan Zhao, et al. 2022. Network Intrusion Detection Combined Hybrid Sampling With Deep Hierarchical Network. *IEEE Access* 10 (2022).

[16] Sayash Kapoor and Arvind Narayanan. 2021. (Ir)Reproducible Machine Learning: A Case Study. https://reproducible.cs.princeton.edu, 6 pages. https://reproducible.cs.princeton.edu/

[17] Latifur Khan, Mamoun Awad, and Bhavani Thuraisingham. 2007. A new intrusion detection system using support vector machines and hierarchical clustering. *The VLDB journal* 16, 4 (2007), 507–521.

[18] l7-filter project. 2026. Application Layer Packet Classifier for Linux. https://l7-filter.sourceforge.net/. Accessed 2026-04-08.

[19] Anh Le, Janus Varmarken, Simon Langhoff, Anastasia Shuba, Minas Gjoka, and Athina Markopoulou. 2015. AntMonitor: A system for monitoring from mobile devices. In *Proceedings of the 2015 ACM SIGCOMM Workshop on Crowdsourcing and Crowdsharing of Big (Internet) Data*. 15–20.

[20] Richard P Lippmann, David J Fried, Isaac Graf, Joshua W Haines, Kristopher R Kendall, David McClung, Dan Weber, Seth E Webster, Dan Wyschogrod, Robert K Cunningham, et al. 2000. Evaluating intrusion detection systems: The 1998 DARPA off-line intrusion detection evaluation. In *Proceedings DARPA Information Survivability Conference and Exposition. DISCEX'00*, Vol. 2. IEEE, 12–26.

[21] Hongyu Liu and Bo Lang. 2022. Machine Learning and Deep Learning Methods for Intrusion Detection Systems: A Survey. *Applied Sciences* 12, 2 (2022).

[22] Mohammad Lotfollahi, Mahdi Jafari Siavoshani, Ramin Shirali Hossein Zade, and Mohammdsadegh Saberian. 2020. Deep packet: A novel approach for encrypted traffic classification using deep learning. *Soft Computing* 24, 3 (2020), 1999–2012.

[23] Gordon Fyodor Lyon. 2009. *Nmap Network Scanning: The Official Nmap Project Guide to Network Discovery and Security Scanning*. Insecure, USA.

[24] Samer Moualla, Khaldoun Khorzom, and Assef Jafar. 2021. Improving the Performance of Machine Learning-Based Network Intrusion Detection Systems on the UNSW-NB15 Dataset. *Computational Intelligence and Neuroscience* 2021 (2021).

[25] Nour Moustafa and Jill Slay. 2015. UNSW-NB15: a comprehensive data set for network intrusion detection systems (UNSW-NB15 network data set). In *2015 military communications and information systems conference (MilCIS)*. IEEE, 1–6.

[26] Biswanath Mukherjee, L Todd Heberlein, and Karl N Levitt. 1994. Network intrusion detection. *IEEE network* 8, 3 (1994), 26–41.

[27] Muhammad Bisri Musthafa, Samsul Huda, Yuta Kodera, Md Arshad Ali, Shunsuke Araki, Jedidah Mwaura, and Yasuyuki Nogami. 2024. Optimizing IoT Intrusion Detection Using Balanced Class Distribution, Feature Selection, and Ensemble Machine Learning Techniques. *Sensors* 24, 14 (2024).

[28] Andriy Panchenko, Lukas Niessen, Andreas Zinnen, and Thomas Engel. 2011. Website fingerprinting in onion routing based anonymization networks. In *Proceedings of the 10th annual ACM workshop on Privacy in the electronic society*. 103–114.

[29] Michael Roberts, Derek Driggs, Matthew Thorpe, Julian Gilbey, Michael Yeung, Stephan Ursprung, Angelica I Aviles-Rivero, Christian Etmann, Cathal McCague, Lucian Beer, et al. 2021. Common pitfalls and recommendations for using machine learning to detect and prognosticate for COVID-19 using chest radiographs and CT scans. *Nature Machine Intelligence* 3, 3 (2021), 199–217.

[30] Mohanad Sarhan, Siamak Layeghy, Nour Moustafa, and Marius Portmann. 2022. Towards a Standard Feature Set for Network Intrusion Detection System Datasets. *Mobile Networks and Applications* 27 (2022), 357–370.

[31] Tal Shapira and Yuval Shavitt. 2021. FlowPic: A Generic Representation for Encrypted Traffic Classification and Applications Identification. *IEEE Transactions on Network and Service Management* 18, 2 (2021), 2052–2064.

[32] Iman Sharafaldin, Arash Habibi Lashkari, and Ali A Ghorbani. 2018. Toward generating a new intrusion detection dataset and intrusion traffic characterization. *ICISSp* 1 (2018), 108–116.

[33] Sandra Siby, Marc Juarez, Claudia Diaz, Narseo Vallina-Rodriguez, and Carmela Troncoso. 2019. Encrypted DNS–> Privacy? A traffic analysis perspective. *arXiv preprint arXiv:1906.09682* (2019).

[34] tcpdump. 2021. *Man page of TCPDUMP*. https://www.tcpdump.org/manpages/tcpdump.1.html

[35] The nPrint Project. 2026. pcapML. https://github.com/nprint/pcapml. Accessed 2026-06-12.

[36] Radu Velea, Casian Ciobanu, Florina Gurzau, and Victor-Valeriu Patriciu. 2017. Feature extraction and visualization for network pcapng traces. In *2017 21st International Conference on Control Systems and Computer Science (CSCS)*. IEEE, 311–316.

[37] Radu Velea, Casian Ciobanu, Laurențiu Margarit, and Ion Bica. 2017. Network traffic anomaly detection using shallow packet inspection and parallel k-means data clustering. *Studies in Informatics and Control* 26, 4 (2017), 387–396.

[38] Pratik Waghmode, Manideep Kanumuri, Hosam El-Ocla, and Tanner Boyle. 2025. Intrusion detection system based on machine learning using least square support vector machine. *Scientific Reports* 15 (2025).

[39] Wei Wang, Yiqiang Sheng, Jinlin Wang, Xuewen Zeng, Xiaozhou Ye, Yongzhong Huang, and Ming Zhu. 2017. HAST-IDS: Learning hierarchical spatial-temporal features using deep neural networks to improve intrusion detection. *IEEE access* 6 (2017), 1792–1806.

[40] Wei Wang, Ming Zhu, Jinlin Wang, Xuewen Zeng, and Zhongzhen Yang. 2017. End-to-end encrypted traffic classification with one-dimensional convolution neural networks. In *2017 IEEE International Conference on Intelligence and Security Informatics (ISI)*. IEEE, 43–48.

[41] Wireshark. 2020. *PcapNG*. https://gitlab.com/wireshark/wireshark/-/wikis/Development/PcapNg

[42] Yong Zhang, Xu Chen, Lei Jin, Xiaojuan Wang, and Da Guo. 2019. Network intrusion detection: Based on deep hierarchical network and original flow data. *IEEE Access* 7 (2019), 37004–37016.